SKILLARGER

Learn. Build. Get Hired.

DATA SCIENCE WITH PYTHON

Pandas Practice Set

150 Questions | Concepts + Solutions + Outputs

50 Basic + 50 Medium + 50 Advanced

Selection | GroupBy | Merge | Reshaping | Time Series | Cleaning | Mini Projects

Copyright SKILLARGER. For learning use by SKILLARGER students.

How to Use This Practice Set

This practice set is designed for the Pandas module of the **SKILLARGER DATA SCIENCE WITH PYTHON** course. Every question shows a **Concepts Used** box (the Pandas tools needed to solve it), a solution and the **actual output**. All 150 solutions were executed and verified.

Section	Questions	Level	What you practise
A	Q1 - Q50	Basic	Inspection, selection, filtering, sorting, new columns, basic groupby, missing values, strings, dates
B	Q51 - Q100	Medium	Named aggregation, transform, pivot, merge/join, reshape, datetime, rolling, cleaning, encoding
C	Q101 - Q150	Advanced	Window functions, merge_asof, cohorts, RFM, sessionization, time zones, method chaining, mini projects

Study Tips

- Read the Concepts Used box first. If you know those tools, try to solve the question yourself before reading the solution.
- Run the dataset setup code (next page) once in Jupyter / Google Colab, then try every question on those same DataFrames.
- Solutions that change data always work on a .copy(), so the original DataFrames stay intact for the next question.
- Outputs were generated with Python 3 and pandas 3.0.x. On older pandas versions you may see tiny display differences (for example datetime64[ns] instead of datetime64[us]).

Practice Datasets (Run This First)

Every question uses these small, ready-made DataFrames. Copy the code below into your notebook and run it once.

DataFrame	Rows x Cols	Description
emp	10 x 8	Employee records (dept, city, age, salary, experience, join date)
sales	12 x 8	Sales orders (date, customer, product, category, region, qty, price)
cust	6 x 4	Customer master (name, segment, city) - C6 has no orders
dirty	6 x 4	Messy data: spaces, wrong case, text numbers, NaN
dup	6 x 2	Table with duplicate ids
scores	3 x 4	Wide marks table (one column per subject)
ts	30 x 3	Daily price and volume series (30 days)

SETUP CODE

```
import pandas as pd, numpy as np
from io import StringIO

# 1) Employee data
emp = pd.DataFrame({
    'emp_id': range(101, 111),
    'name': ['Aarav','Diya','Kabir','Meera','Rohan','Sana','Vikram','Isha','Neel','Tara'],
    'dept': ['IT','HR','IT','Finance','Sales','Finance','Sales','IT','HR','Sales'],
    'city': ['Pune','Delhi','Mumbai','Pune','Delhi','Mumbai','Pune','Delhi','Mumbai','Pune'],
    'age': [28,34,25,41,30,29,38,26,45,32],
    'salary': [65000,48000,72000,90000,52000,61000,58000,69000,75000,47000],
    'exp': [4,9,2,15,6,5,12,3,18,7],
    'join_date': pd.to_datetime([
        '2020-03-15', '2016-07-01', '2022-01-10', '2011-05-20', '2018-09-05',
        '2019-11-25', '2014-02-14', '2021-06-30', '2008-12-01', '2017-04-18'])})

# 2) Sales orders
sales = pd.DataFrame({
    'order_id': range(1, 13),
    'order_date': pd.to_datetime([
        '2024-01-05', '2024-01-12', '2024-01-20', '2024-02-03', '2024-02-14', '2024-02-25',
        '2024-03-02', '2024-03-15', '2024-03-28', '2024-04-04', '2024-04-18', '2024-04-30']),
    'customer_id': ['C1','C2','C1','C3','C2','C4','C1','C3','C5','C4','C2','C5'],
    'product': ['Laptop','Mouse','Keyboard','Monitor','Laptop','Mouse',
                'Monitor','Keyboard','Laptop','Mouse','Monitor','Keyboard'],
    'category': ['Computers','Accessories','Accessories','Displays','Computers','Accessories',
                 'Displays','Accessories','Computers','Accessories','Displays','Accessories'],
    'region': ['North','South','North','East','West','South',
               'North','East','West','North','South','East'],
    'qty': [2,10,5,3,1,8,2,4,3,12,1,6],
    'price': [55000,500,1200,12000,56000,520,
              12500,1150,54000,480,12200,1250]})

# 3) Customers
cust = pd.DataFrame({
    'customer_id': ['C1','C2','C3','C4','C5','C6'],
    'cust_name': ['Rahul','Anita','Suresh','Pooja','Manoj','Kavya'],
    'segment': ['Retail','Corporate','Retail','Corporate','Retail','Corporate'],
    'city': ['Pune','Delhi','Mumbai','Chennai','Kolkata','Pune']})

# 4) Messy data and duplicates
dirty = pd.DataFrame({
    'name': [' alice ', 'BOB', 'Charlie', 'bob', None, 'Eve '],
```

```
    'age': [25, np.nan, 31, 28, 40, np.nan],
    'score': ['85', '90', 'x', '78', '66', '90'],
    'city': ['pune', 'Delhi', 'Mumbai', 'delhi', 'Pune', None]})
dup = pd.DataFrame({'id': [1, 2, 2, 3, 3, 3], 'val': [10, 20, 20, 30, 35, 30]})

# 5) Wide marks table
scores = pd.DataFrame({'student': ['A', 'B', 'C'], 'math': [80, 90, 70],
                       'sci': [75, 85, 95], 'eng': [88, 72, 60]})

# 6) Daily time series
rng = np.random.default_rng(0)
ts = pd.DataFrame({'date': pd.date_range('2024-01-01', periods=30),
                   'price': (100 + rng.normal(0, 2, 30).cumsum()).round(2),
                   'volume': rng.integers(1000, 5000, 30)})

pd.set_option('display.width', 90)
pd.set_option('display.max_columns', 20)
```

SECTION A - BASIC LEVEL

Q1 to Q50 | Foundations: load, inspect, select, filter, sort, group, clean

BASIC | DATAFRAME INSPECTION

Q1. Display the first 3 rows of the emp DataFrame, then print its shape (rows, columns) and the list of column names.

CONCEPTS USED: head(), shape, columns, len()

SOLUTION

```
print(emp.head(3))
print(emp.shape)
print(emp.columns.tolist())
```

OUTPUT

```
   emp_id   name dept    city  age  salary  exp  join_date
0     101  Aarav   IT    Pune   28   65000    4 2020-03-15
1     102   Diya   HR   Delhi   34   48000    9 2016-07-01
2     103  Kabir   IT  Mumbai   25   72000    2 2022-01-10
(10, 8)
['emp_id', 'name', 'dept', 'city', 'age', 'salary', 'exp', 'join_date']
```

BASIC | DATAFRAME INSPECTION

Q2. Show the data type of every column of emp and a statistical summary (count, mean, std, quartiles) of the numeric columns age, salary and exp.

CONCEPTS USED: dtypes, describe(), round()

SOLUTION

```
print(emp.dtypes)
print(emp[['age', 'salary', 'exp']].describe().round(1))
```

OUTPUT

```
emp_id               int64
name                   str
dept                   str
city                   str
age                  int64
salary               int64
exp                  int64
join_date    datetime64[us]
dtype: object
        age   salary   exp
count  10.0     10.0  10.0
mean   32.8  63700.0   8.1
std     6.6  13416.8   5.3
min    25.0  47000.0   2.0
25%    28.2  53500.0   4.2
50%    31.0  63000.0   6.5
75%    37.0  71250.0  11.2
max    45.0  90000.0  18.0
```

BASIC | SELECTING COLUMNS

Q3. Select the single column 'name' (a Series) and then the two columns 'name' and 'salary' (a DataFrame). Show only the first 3 rows of each.

CONCEPTS USED: df['col'], df[['c1','c2']], Series vs DataFrame

SOLUTION

```
print(emp['name'].head(3))
print(type(emp['name']).__name__)
print(emp[['name', 'salary']].head(3))
```

OUTPUT

```
0    Aarav
1     Diya
2    Kabir
Name: name, dtype: str
Series
    name  salary
0  Aarav   65000
1   Diya   48000
2  Kabir   72000
```

BASIC | POSITION-BASED SELECTION

Q4. Using iloc, select: (a) the 3rd row, (b) rows 0 to 2 and columns 1 to 3, (c) rows 0 and 5 only.

CONCEPTS USED: iloc, row/column slicing, list of positions

SOLUTION

```
print(emp.iloc[2])
print(emp.iloc[0:3, 1:4])
print(emp.iloc[[0, 5]])
```

OUTPUT

```
emp_id                      103
name                      Kabir
dept                         IT
city                     Mumbai
age                          25
salary                    72000
exp                           2
join_date    2022-01-10 00:00:00
Name: 2, dtype: object
    name dept    city
0  Aarav   IT    Pune
1   Diya   HR   Delhi
2  Kabir   IT  Mumbai
   emp_id   name     dept    city  age  salary  exp  join_date
0     101  Aarav       IT    Pune   28   65000    4 2020-03-15
5     106   Sana  Finance  Mumbai   29   61000    5 2019-11-25
```

BASIC | LABEL-BASED SELECTION

Q5. Using loc, select rows with index 2 to 4 (inclusive) and only the columns 'name' and 'salary'.

CONCEPTS USED: loc, inclusive slice, column list

SOLUTION

```
print(emp.loc[2:4, ['name', 'salary']])
```

OUTPUT

```
    name  salary
2  Kabir   72000
3  Meera   90000
4  Rohan   52000
```

BASIC | FILTERING ROWS

Q6. Find all employees whose salary is greater than 60000.

CONCEPTS USED: boolean mask, comparison operators

SOLUTION

```
print(emp[emp['salary'] > 60000][['name', 'dept', 'salary']])
```

OUTPUT

```
    name     dept  salary
0  Aarav       IT   65000
2  Kabir       IT   72000
3  Meera  Finance   90000
5   Sana  Finance   61000
7   Isha       IT   69000
8   Neel       HR   75000
```

BASIC | FILTERING ROWS

Q7. Find employees who work in the 'IT' department AND earn more than 65000, and separately those who are in 'HR' OR 'Finance'.

CONCEPTS USED: & (and), | (or), parentheses

SOLUTION

```
print(emp[(emp['dept'] == 'IT') & (emp['salary'] > 65000)][['name', 'dept', 'salary']])
print(emp[(emp['dept'] == 'HR') | (emp['dept'] == 'Finance')][['name', 'dept']])
```

OUTPUT

```
    name dept  salary
2  Kabir   IT   72000
7   Isha   IT   69000
    name     dept
1   Diya       HR
3  Meera  Finance
5   Sana  Finance
8   Neel       HR
```

BASIC | FILTERING ROWS

Q8. Find employees living in Pune or Delhi using isin(), and then employees NOT living in those cities.

CONCEPTS USED: isin(), ~ (not)

SOLUTION

```
print(emp[emp['city'].isin(['Pune', 'Delhi'])]['name'].tolist())
print(emp[~emp['city'].isin(['Pune', 'Delhi'])]['name'].tolist())
```

OUTPUT

```
['Aarav', 'Diya', 'Meera', 'Rohan', 'Vikram', 'Isha', 'Tara']
['Kabir', 'Sana', 'Neel']
```

BASIC | FILTERING ROWS

Q9. Find employees whose age is between 28 and 35 (both inclusive).

CONCEPTS USED: between()

SOLUTION

```
print(emp[emp['age'].between(28, 35)][['name', 'age']])
```

OUTPUT

```
    name  age
0  Aarav   28
1   Diya   34
4  Rohan   30
5   Sana   29
9   Tara   32
```

BASIC | STRING FILTERING

Q10. Find employees whose name starts with 'A' or 'S', and employees whose name contains the letter 'i'.

CONCEPTS USED: str.startswith(), str.contains()

SOLUTION

```
print(emp[emp['name'].str.startswith(('A', 'S'))]['name'].tolist())
print(emp[emp['name'].str.contains('i')]['name'].tolist())
```

OUTPUT

```
['Aarav', 'Sana']
['Diya', 'Kabir', 'Vikram']
```

BASIC | CREATING COLUMNS

Q11. Add a new column 'bonus' equal to 10% of salary and another column 'total_pay' = salary + bonus.

CONCEPTS USED: new column from arithmetic

SOLUTION

```
df = emp.copy()
df['bonus'] = df['salary'] * 0.10
df['total_pay'] = df['salary'] + df['bonus']
print(df[['name', 'salary', 'bonus', 'total_pay']].head())
```

OUTPUT

```
    name  salary   bonus  total_pay
0  Aarav   65000  6500.0    71500.0
1   Diya   48000  4800.0    52800.0
2  Kabir   72000  7200.0    79200.0
3  Meera   90000  9000.0    99000.0
4  Rohan   52000  5200.0    57200.0
```

BASIC | CONDITIONAL COLUMNS

Q12. Create a column 'level' that is 'Senior' if exp >= 10, otherwise 'Junior'.

CONCEPTS USED: np.where()

SOLUTION

```
df = emp.copy()
df['level'] = np.where(df['exp'] >= 10, 'Senior', 'Junior')
print(df[['name', 'exp', 'level']])
```

OUTPUT

```
     name  exp   level
0   Aarav    4  Junior
1    Diya    9  Junior
2   Kabir    2  Junior
3   Meera   15  Senior
4   Rohan    6  Junior
5    Sana    5  Junior
6  Vikram   12  Senior
7    Isha    3  Junior
8    Neel   18  Senior
9    Tara    7  Junior
```

BASIC | RENAMING

Q13. Rename the columns 'emp_id' to 'ID' and 'salary' to 'CTC'. Show the new column list.

CONCEPTS USED: rename(columns=...)

SOLUTION

```
df = emp.rename(columns={'emp_id': 'ID', 'salary': 'CTC'})
print(df.columns.tolist())
```

OUTPUT

```
['ID', 'name', 'dept', 'city', 'age', 'CTC', 'exp', 'join_date']
```

BASIC | DROPPING

Q14. Drop the 'city' and 'join_date' columns, and also drop the first two rows (index 0 and 1).

CONCEPTS USED: drop(columns=...), drop(index=...)

SOLUTION

```
df = emp.drop(columns=['city', 'join_date']).drop(index=[0, 1])
print(df.head(3))
print(df.shape)
```

OUTPUT

```
   emp_id   name     dept  age  salary  exp
2     103  Kabir       IT   25   72000    2
3     104  Meera  Finance   41   90000   15
4     105  Rohan    Sales   30   52000    6
(8, 6)
```

BASIC | SORTING

Q15. Sort employees by salary in descending order and show the top 3 (name and salary).

CONCEPTS USED: sort_values(), ascending=False

SOLUTION

```
print(emp.sort_values('salary', ascending=False)[['name', 'salary']].head(3))
```

OUTPUT

```
    name  salary
3  Meera   90000
8   Neel   75000
2  Kabir   72000
```

BASIC | SORTING

Q16. Sort by department (A-Z) and then by salary (high to low) within each department.

CONCEPTS USED: sort_values() with multiple columns

SOLUTION

```
print(emp.sort_values(['dept', 'salary'], ascending=[True, False])[['dept', 'name', 'salary']])
```

OUTPUT

```
      dept    name  salary
3  Finance   Meera   90000
5  Finance    Sana   61000
8       HR    Neel   75000
1       HR    Diya   48000
2       IT   Kabir   72000
7       IT    Isha   69000
0       IT   Aarav   65000
6    Sales  Vikram   58000
4    Sales   Rohan   52000
9    Sales    Tara   47000
```

BASIC | TOP / BOTTOM N

Q17. Find the 3 oldest employees and the 2 lowest-paid employees.

CONCEPTS USED: nlargest(), nsmallest()

SOLUTION

```
print(emp.nlargest(3, 'age')[['name', 'age']])
print(emp.nsmallest(2, 'salary')[['name', 'salary']])
```

OUTPUT

```
     name  age
8    Neel   45
3   Meera   41
6  Vikram   38
   name  salary
9  Tara   47000
1  Diya   48000
```

BASIC | COUNTING VALUES

Q18. Count how many employees are in each department, and also show it as percentages.

CONCEPTS USED: value_counts(), normalize=True

SOLUTION

```
print(emp['dept'].value_counts())
print(emp['dept'].value_counts(normalize=True).round(2))
```

OUTPUT

```
dept
IT         3
Sales      3
HR         2
Finance    2
Name: count, dtype: int64
dept
IT         0.3
Sales      0.3
HR         0.2
Finance    0.2
Name: proportion, dtype: float64
```

BASIC | UNIQUE VALUES

Q19. Find the unique cities in emp and how many distinct cities there are.

CONCEPTS USED: unique(), nunique()

SOLUTION

```
print(emp['city'].unique())
print(emp['city'].nunique())
```

OUTPUT

```
<StringArray>
['Pune', 'Delhi', 'Mumbai']
Length: 3, dtype: str
3
```

BASIC | BASIC AGGREGATION

Q20. Calculate the total, average, maximum, minimum and median of the salary column.

CONCEPTS USED: sum(), mean(), max(), min(), median()

SOLUTION

```
s = emp['salary']
print(s.sum(), s.mean(), s.max(), s.min(), s.median())
```

OUTPUT

```
637000 63700.0 90000 47000 63000.0
```

BASIC | GROUPBY BASICS

Q21. Find the average salary of each department.

CONCEPTS USED: groupby(), mean()

SOLUTION

```
print(emp.groupby('dept')['salary'].mean())
```

OUTPUT

```
dept
Finance    75500.000000
HR         61500.000000
IT         68666.666667
Sales      52333.333333
Name: salary, dtype: float64
```

BASIC | GROUPBY BASICS

Q22. Count the number of employees in each city.

CONCEPTS USED: groupby(), size()/count()

SOLUTION

```
print(emp.groupby('city').size())
```

OUTPUT

```
city
Delhi     3
Mumbai    3
Pune      4
dtype: int64
```

BASIC | GROUPBY AGGREGATION

Q23. For each department, show the minimum, maximum and mean salary in one table.

CONCEPTS USED: groupby(), agg() with a list of functions

SOLUTION

```
print(emp.groupby('dept')['salary'].agg(['min', 'max', 'mean']))
```

OUTPUT

```
            min    max          mean
dept
Finance  61000  90000  75500.000000
HR       48000  75000  61500.000000
IT       65000  72000  68666.666667
Sales    47000  58000  52333.333333
```

BASIC | FINDING INDEX OF MAX/MIN

Q24. Find the name of the highest-paid employee and the name of the youngest employee.

CONCEPTS USED: idxmax(), idxmin(), loc

SOLUTION

```
print(emp.loc[emp['salary'].idxmax(), 'name'])
print(emp.loc[emp['age'].idxmin(), 'name'])
```

OUTPUT

```
Meera
Kabir
```

BASIC | INDEX OPERATIONS

Q25. Set 'emp_id' as the index, select the employee with ID 105, then reset the index back.

CONCEPTS USED: set_index(), reset_index()

SOLUTION

```
df = emp.set_index('emp_id')
print(df.loc[105, ['name', 'dept']])
print(df.reset_index().columns.tolist()[:3])
```

OUTPUT

```
name    Rohan
dept    Sales
Name: 105, dtype: str
['emp_id', 'name', 'dept']
```

BASIC | MISSING VALUES

Q26. In the dirty DataFrame, count the missing values in each column and the percentage of missing values.

CONCEPTS USED: isna(), sum(), isna().mean()

SOLUTION

```
print(dirty.isna().sum())
print((dirty.isna().mean() * 100).round(1))
```

OUTPUT

```
name     1
age      2
score    0
city     1
dtype: int64
name     16.7
age      33.3
score     0.0
city     16.7
dtype: float64
```

BASIC | FILLING MISSING VALUES

Q27. Fill the missing 'age' values in dirty with the mean age and the missing 'city' values with 'Unknown'.

CONCEPTS USED: fillna(), mean()

SOLUTION

```
df = dirty.copy()
df['age'] = df['age'].fillna(df['age'].mean())
df['city'] = df['city'].fillna('Unknown')
print(df[['age', 'city']])
```

OUTPUT

```
    age     city
0  25.0     pune
1  31.0    Delhi
2  31.0   Mumbai
3  28.0    delhi
4  40.0     Pune
5  31.0  Unknown
```

BASIC | DROPPING MISSING VALUES

Q28. Drop all rows of dirty that contain any missing value, and separately drop rows where only 'name' is missing.

CONCEPTS USED: dropna(), subset=

SOLUTION

```
print(dirty.dropna().shape)
print(dirty.dropna(subset=['name']).shape)
```

OUTPUT

```
(3, 4)
(5, 4)
```

BASIC | DUPLICATES

Q29. In dup, find the duplicated rows, then remove duplicates keeping the last row for each 'id'.

CONCEPTS USED: duplicated(), drop_duplicates(subset, keep)

SOLUTION

```
print(dup.duplicated().tolist())
print(dup.drop_duplicates(subset='id', keep='last'))
```

OUTPUT

```
[False, False, True, False, False, True]
   id  val
0   1   10
2   2   20
5   3   30
```

BASIC | TYPE CONVERSION

Q30. The 'score' column of dirty is stored as text and has a bad value 'x'. Convert it to numbers, turning bad values into NaN.

CONCEPTS USED: pd.to_numeric(errors='coerce'), astype()

SOLUTION

```
df = dirty.copy()
df['score'] = pd.to_numeric(df['score'], errors='coerce')
print(df['score'].tolist())
print(df['score'].dtype)
```

OUTPUT

```
[85.0, 90.0, nan, 78.0, 66.0, 90.0]
float64
```

BASIC | STRING CLEANING

Q31. Clean the 'name' column of dirty: remove extra spaces and convert to Title Case.

CONCEPTS USED: str.strip(), str.lower(), str.title()

SOLUTION

```
print(dirty['name'].str.strip().str.title().tolist())
```

OUTPUT

```
['Alice', 'Bob', 'Charlie', 'Bob', nan, 'Eve']
```

BASIC | APPLY()

Q32. Convert the salary of every employee into lakhs (divide by 100000) using apply with a lambda.

CONCEPTS USED: apply() with lambda

SOLUTION

```
lakhs = emp['salary'].apply(lambda x: x / 100000)
print(lakhs.head())
```

OUTPUT

```
0    0.65
1    0.48
2    0.72
3    0.90
4    0.52
Name: salary, dtype: float64
```

BASIC | MAP()

Q33. Map the departments to codes: IT -> 'D1', HR -> 'D2', Finance -> 'D3', Sales -> 'D4' in a new column.

CONCEPTS USED: map() with a dictionary

SOLUTION

```
df = emp.copy()
df['dept_code'] = df['dept'].map({'IT': 'D1', 'HR': 'D2', 'Finance': 'D3', 'Sales': 'D4'})
print(df[['name', 'dept', 'dept_code']].head())
```

OUTPUT

```
    name     dept dept_code
0  Aarav       IT        D1
1   Diya       HR        D2
2  Kabir       IT        D1
3  Meera  Finance        D3
4  Rohan    Sales        D4
```

BASIC | DATETIME BASICS

Q34. From the join_date column extract the year, the month name and the weekday name.

CONCEPTS USED: dt.year, dt.month_name(), dt.day_name()

SOLUTION

```
d = emp['join_date']
out = pd.DataFrame({'year': d.dt.year, 'month': d.dt.month_name(), 'day': d.dt.day_name()})
print(out.head())
```

OUTPUT

```
   year      month        day
0  2020      March     Sunday
1  2016       July     Friday
2  2022    January     Monday
3  2011        May     Friday
4  2018  September  Wednesday
```

BASIC | DATETIME FILTERING

Q35. Find employees who joined after 1st January 2019.

CONCEPTS USED: boolean mask on datetime, pd.Timestamp

SOLUTION

```
print(emp[emp['join_date'] > '2019-01-01'][['name', 'join_date']])
```

OUTPUT

```
    name  join_date
0  Aarav 2020-03-15
2  Kabir 2022-01-10
5   Sana 2019-11-25
7   Isha 2021-06-30
```

BASIC | COLUMN ARITHMETIC

Q36. Calculate 'salary per year of experience' for each employee, rounded to 0 decimals.

CONCEPTS USED: vectorized column operations, round()

SOLUTION

```
df = emp.copy()
df['sal_per_exp'] = (df['salary'] / df['exp']).round(0)
print(df[['name', 'salary', 'exp', 'sal_per_exp']].head())
```

OUTPUT

```
    name  salary  exp  sal_per_exp
0  Aarav   65000    4      16250.0
1   Diya   48000    9       5333.0
2  Kabir   72000    2      36000.0
3  Meera   90000   15       6000.0
4  Rohan   52000    6       8667.0
```

BASIC | CUMULATIVE FUNCTIONS

Q37. Add a running total (cumulative sum) of the 'qty' column in sales.

CONCEPTS USED: cumsum(), cummax()

SOLUTION

```
df = sales[['order_id', 'qty']].copy()
df['running_qty'] = df['qty'].cumsum()
print(df.head(6))
```

OUTPUT

```
   order_id  qty  running_qty
0         1    2            2
1         2   10           12
2         3    5           17
3         4    3           20
4         5    1           21
5         6    8           29
```

BASIC | REVENUE CALCULATION

Q38. Create a 'revenue' column (qty * price) in sales and compute total revenue.

CONCEPTS USED: new column, sum()

SOLUTION

```
df = sales.copy()
df['revenue'] = df['qty'] * df['price']
print(df[['order_id', 'product', 'revenue']].head())
print(df['revenue'].sum())
```

OUTPUT

```
   order_id   product  revenue
0         1    Laptop   110000
1         2     Mouse     5000
2         3  Keyboard     6000
3         4   Monitor    36000
4         5    Laptop    56000
434220
```

BASIC | GROUPBY SUM

Q39. Find the total revenue per product category, sorted from highest to lowest.

CONCEPTS USED: groupby(), sum(), sort_values()

SOLUTION

```
df = sales.assign(revenue=sales['qty'] * sales['price'])
print(df.groupby('category')['revenue'].sum().sort_values(ascending=False))
```

OUTPUT

```
category
Computers      328000
Displays        73200
Accessories     33020
Name: revenue, dtype: int64
```

BASIC | SELECTING BY DTYPE

Q40. Select only the numeric columns of emp, and then only the text (object/string) columns.

CONCEPTS USED: select_dtypes()

SOLUTION

```
print(emp.select_dtypes(include='number').columns.tolist())
print(emp.select_dtypes(exclude=['number', 'datetime']).columns.tolist())
```

OUTPUT

```
['emp_id', 'age', 'salary', 'exp']
['name', 'dept', 'city']
```

BASIC | QUERY()

Q41. Using query(), find Sales department employees older than 30.

CONCEPTS USED: query() with strings and variables

SOLUTION

```
print(emp.query("dept == 'Sales' and age > 30")[['name', 'dept', 'age']])
```

OUTPUT

```
     name   dept  age
6  Vikram  Sales   38
9    Tara  Sales   32
```

BASIC | COUNTING CONDITIONS

Q42. How many employees earn more than 60000, and what percentage of all employees is that?

CONCEPTS USED: boolean sum(), mean()

SOLUTION

```
m = emp['salary'] > 60000
print(m.sum(), m.mean() * 100)
```

OUTPUT

```
6 60.0
```

BASIC | SAMPLING

Q43. Pick 3 random employees (reproducible) and also show the last 2 rows of emp.

CONCEPTS USED: sample(random_state), head(), tail()

SOLUTION

```
print(emp.sample(3, random_state=1)[['name', 'dept']])
print(emp.tail(2)[['name', 'dept']])
```

OUTPUT

```
     name   dept
2   Kabir     IT
9    Tara  Sales
6  Vikram  Sales
   name   dept
8  Neel     HR
9  Tara  Sales
```

BASIC | COPY()

Q44. Create a separate DataFrame of IT employees and add a 'bonus' column to it without affecting emp.

CONCEPTS USED: copy(), avoiding SettingWithCopyWarning

SOLUTION

```
it = emp[emp['dept'] == 'IT'].copy()
it['bonus'] = it['salary'] * 0.05
print(it[['name', 'salary', 'bonus']])
print('bonus' in emp.columns)
```

OUTPUT

```
    name  salary   bonus
0  Aarav   65000  3250.0
2  Kabir   72000  3600.0
7   Isha   69000  3450.0
False
```

BASIC | CORRELATION

Q45. Find the correlation matrix between age, salary and exp.

CONCEPTS USED: corr(), round()

SOLUTION

```
print(emp[['age', 'salary', 'exp']].corr().round(2))
```

OUTPUT

```
          age  salary   exp
age     1.00    0.32  1.00
salary  0.32    1.00  0.35
exp     1.00    0.35  1.00
```

BASIC | CREATING DATAFRAMES

Q46. Create a DataFrame from a list of dictionaries (3 students with name and marks) and then add a Grade column.

CONCEPTS USED: pd.DataFrame from list of dicts

SOLUTION

```
data = [{'name': 'Ravi', 'marks': 82},
        {'name': 'Sita', 'marks': 91},
        {'name': 'Tom', 'marks': 67}]
df = pd.DataFrame(data)
df['grade'] = np.where(df['marks'] >= 80, 'A', 'B')
print(df)
```

OUTPUT

```
   name  marks grade
0  Ravi     82     A
1  Sita     91     A
2   Tom     67     B
```

BASIC | SERIES BASICS

Q47. Create a Series of marks with subject names as the index, access one subject by label and add 5 bonus marks to all.

CONCEPTS USED: pd.Series with index, label access, arithmetic

SOLUTION

```
s = pd.Series([78, 85, 92], index=['Math', 'Science', 'English'])
print(s['Science'])
print(s + 5)
```

OUTPUT

```
85
Math       83
Science    90
English    97
dtype: int64
```

BASIC | BINNING

Q48. Group employees into age groups: 20-30 'Young', 30-40 'Mid', 40-50 'Senior' using pd.cut.

CONCEPTS USED: pd.cut() with bins and labels

SOLUTION

```
df = emp.copy()
df['age_group'] = pd.cut(df['age'], bins=[20, 30, 40, 50], labels=['Young', 'Mid', 'Senior'])
print(df[['name', 'age', 'age_group']].head())
print(df['age_group'].value_counts())
```

OUTPUT

```
    name  age age_group
0  Aarav   28     Young
1   Diya   34       Mid
2  Kabir   25     Young
3  Meera   41    Senior
4  Rohan   30     Young
age_group
Young     5
Mid       3
Senior    2
Name: count, dtype: int64
```

BASIC | STRING METHODS

Q49. Create company email IDs like 'aarav@skillarger.com' for every employee and compute the length of each name.

CONCEPTS USED: str.len(), str.lower(), string concatenation

SOLUTION

```
df = emp.copy()
df['email'] = df['name'].str.lower() + '@skillarger.com'
df['name_len'] = df['name'].str.len()
print(df[['name', 'email', 'name_len']].head(4))
```

OUTPUT

```
    name                email  name_len
0  Aarav  aarav@skillarger.com         5
1   Diya   diya@skillarger.com         4
2  Kabir  kabir@skillarger.com         5
3  Meera  meera@skillarger.com         5
```

BASIC | READING / WRITING CSV

Q50. Convert emp (first 3 rows, selected columns) to CSV text and read it back into a DataFrame.

CONCEPTS USED: to_csv(), read_csv(), StringIO

SOLUTION

```
csv_text = emp[['name', 'dept', 'salary']].head(3).to_csv(index=False)
print(csv_text)
back = pd.read_csv(StringIO(csv_text))
print(back.dtypes)
```

OUTPUT

```
name,dept,salary
Aarav,IT,65000
Diya,HR,48000
Kabir,IT,72000

name        str
dept        str
salary    int64
dtype: object
```

SECTION B - MEDIUM LEVEL

Q51 to Q100 | Real analysis: join, reshape, transform, dates, rolling, encoding

MEDIUM | NAMED AGGREGATION

Q51. For each department, build a summary table with headcount, average salary and maximum experience using named aggregation.

CONCEPTS USED: groupby(), agg() with named aggregation

SOLUTION

```
res = emp.groupby('dept').agg(headcount=('emp_id', 'count'),
                              avg_salary=('salary', 'mean'),
                              max_exp=('exp', 'max'))
print(res)
```

OUTPUT

```
          headcount    avg_salary  max_exp
dept
Finance           2  75500.000000       15
HR                2  61500.000000       18
IT                3  68666.666667        4
Sales             3  52333.333333       12
```

MEDIUM | GROUPBY TRANSFORM

Q52. Add a column showing how far each employee's salary is from their department's average salary.

CONCEPTS USED: groupby(), transform('mean')

SOLUTION

```
df = emp.copy()
df['dept_avg'] = df.groupby('dept')['salary'].transform('mean')
df['diff'] = (df['salary'] - df['dept_avg']).round(0)
print(df[['name', 'dept', 'salary', 'dept_avg', 'diff']])
```

OUTPUT

```
     name     dept  salary      dept_avg     diff
0   Aarav       IT   65000  68666.666667  -3667.0
1    Diya       HR   48000  61500.000000 -13500.0
2   Kabir       IT   72000  68666.666667   3333.0
3   Meera  Finance   90000  75500.000000  14500.0
4   Rohan    Sales   52000  52333.333333   -333.0
5    Sana  Finance   61000  75500.000000 -14500.0
6  Vikram    Sales   58000  52333.333333   5667.0
7    Isha       IT   69000  68666.666667    333.0
8    Neel       HR   75000  61500.000000  13500.0
9    Tara    Sales   47000  52333.333333  -5333.0
```

MEDIUM | GROUPBY FILTER

Q53. Keep only the employees that belong to departments whose average salary is above 60000.

CONCEPTS USED: groupby().filter()

SOLUTION

```
res = emp.groupby('dept').filter(lambda g: g['salary'].mean() > 60000)
print(res[['name', 'dept', 'salary']])
```

OUTPUT

```
    name     dept  salary
0  Aarav       IT   65000
1   Diya       HR   48000
2  Kabir       IT   72000
3  Meera  Finance   90000
5   Sana  Finance   61000
7   Isha       IT   69000
8   Neel       HR   75000
```

MEDIUM | TOP ROW PER GROUP

Q54. Find the highest-paid employee in each department (name and salary).

CONCEPTS USED: groupby(), idxmax(), loc

SOLUTION

```
idx = emp.groupby('dept')['salary'].idxmax()
print(emp.loc[idx, ['dept', 'name', 'salary']])
```

OUTPUT

```
      dept    name  salary
3  Finance   Meera   90000
8       HR    Neel   75000
2       IT   Kabir   72000
6    Sales  Vikram   58000
```

MEDIUM | RANKING IN GROUPS

Q55. Rank employees by salary within their department (1 = highest paid).

CONCEPTS USED: groupby().rank(method='dense')

SOLUTION

```
df = emp.copy()
df['rank_in_dept'] = df.groupby('dept')['salary'].rank(ascending=False, method='dense')
print(df.sort_values(['dept', 'rank_in_dept'])[['dept', 'name', 'salary', 'rank_in_dept']])
```

OUTPUT

```
      dept    name  salary  rank_in_dept
3  Finance   Meera   90000           1.0
5  Finance    Sana   61000           2.0
8       HR    Neel   75000           1.0
1       HR    Diya   48000           2.0
2       IT   Kabir   72000           1.0
7       IT    Isha   69000           2.0
0       IT   Aarav   65000           3.0
6    Sales  Vikram   58000           1.0
4    Sales   Rohan   52000           2.0
9    Sales    Tara   47000           3.0
```

MEDIUM | PIVOT TABLE

Q56. Create a pivot table of total revenue with regions as rows and categories as columns, including row and column totals.

CONCEPTS USED: pivot_table(), aggfunc, margins

SOLUTION

```
df = sales.assign(revenue=sales['qty'] * sales['price'])
pt = df.pivot_table(index='region', columns='category', values='revenue',
                    aggfunc='sum', fill_value=0, margins=True)
print(pt)
```

OUTPUT

```
category  Accessories  Computers  Displays     All
region
East            12100          0     36000   48100
North           11760     110000     25000  146760
South            9160          0     12200   21360
West                0     218000         0  218000
All             33020     328000     73200  434220
```

MEDIUM | CROSSTAB

Q57. Build a frequency table showing how many employees are in each department for each city.

CONCEPTS USED: pd.crosstab()

SOLUTION

```
print(pd.crosstab(emp['dept'], emp['city']))
```

OUTPUT

```
city     Delhi  Mumbai  Pune
dept
Finance      0       1     1
HR           1       1     0
IT           1       1     1
Sales        1       0     2
```

MEDIUM | MERGE (INNER JOIN)

Q58. Join sales with the customers table to see the customer name and segment for every order.

CONCEPTS USED: merge(), on=, how='inner'

SOLUTION

```
m = sales.merge(cust[['customer_id', 'cust_name', 'segment']], on='customer_id', how='inner')
print(m[['order_id', 'customer_id', 'cust_name', 'segment']].head(6))
print(m.shape)
```

OUTPUT

```
   order_id customer_id cust_name    segment
0         1          C1     Rahul     Retail
1         2          C2     Anita  Corporate
2         3          C1     Rahul     Retail
3         4          C3    Suresh     Retail
4         5          C2     Anita  Corporate
5         6          C4     Pooja  Corporate
(12, 10)
```

MEDIUM | MERGE (LEFT ANTI-JOIN)

Q59. Find the customers who have never placed an order.

CONCEPTS USED: merge(how='left', indicator=True)

SOLUTION

```
m = cust.merge(sales[['customer_id']].drop_duplicates(), on='customer_id',
               how='left', indicator=True)
print(m[m['_merge'] == 'left_only'][['customer_id', 'cust_name']])
```

OUTPUT

```
  customer_id cust_name
5          C6     Kavya
```

MEDIUM | JOIN TYPES

Q60. Compare the number of rows produced by inner, left, right and outer merges between cust and the unique customers in sales.

CONCEPTS USED: merge() how = inner / left / right / outer

SOLUTION

```
s = sales[['customer_id']].drop_duplicates()
for how in ['inner', 'left', 'right', 'outer']:
    print(how, cust.merge(s, on='customer_id', how=how).shape[0])
```

OUTPUT

```
inner 5
left 6
right 5
outer 6
```

MEDIUM | CONCATENATE

Q61. Stack two DataFrames (first 2 and last 2 rows of emp) vertically, then place two column-subsets side by side.

CONCEPTS USED: pd.concat(axis=0 / axis=1)

SOLUTION

```
a, b = emp.head(2), emp.tail(2)
print(pd.concat([a, b])[['name']].T)
side = pd.concat([emp[['name']].head(3), emp[['salary']].head(3)], axis=1)
print(side)
```

OUTPUT

```
          0     1     8     9
name  Aarav  Diya  Neel  Tara
    name  salary
0  Aarav   65000
1   Diya   48000
2  Kabir   72000
```

MEDIUM | JOIN ON INDEX

Q62. Join the employee names with a separate 'rating' Series using the index (emp_id).

CONCEPTS USED: set_index(), join()

SOLUTION

```
rating = pd.Series([4.5, 3.8, 4.1], index=[101, 102, 103], name='rating')
res = emp.set_index('emp_id')[['name']].join(rating)
print(res.head(5))
```

OUTPUT

```
          name  rating
emp_id
101     Aarav     4.5
102      Diya     3.8
103     Kabir     4.1
104     Meera     NaN
105     Rohan     NaN
```

MEDIUM | MERGE DIFFERENT KEYS

Q63. Merge emp with a department-head table where the key column has a different name.

CONCEPTS USED: merge(left_on, right_on)

SOLUTION

```
heads = pd.DataFrame({'department': ['IT', 'HR', 'Finance', 'Sales'],
                      'head': ['Nina', 'Omar', 'Pia', 'Raj']})
m = emp.merge(heads, left_on='dept', right_on='department').drop(columns='department')
print(m[['name', 'dept', 'head']].head(5))
```

OUTPUT

```
    name     dept  head
0  Aarav       IT  Nina
1   Diya       HR  Omar
2  Kabir       IT  Nina
3  Meera  Finance   Pia
4  Rohan    Sales   Raj
```

MEDIUM | MELT (WIDE TO LONG)

Q64. Convert the wide scores table (one column per subject) into long format with columns student, subject, marks.

CONCEPTS USED: melt(id_vars, var_name, value_name)

SOLUTION

```
long = scores.melt(id_vars='student', var_name='subject', value_name='marks')
print(long)
```

OUTPUT

```
  student subject  marks
0       A    math     80
1       B    math     90
2       C    math     70
3       A     sci     75
4       B     sci     85
5       C     sci     95
6       A     eng     88
7       B     eng     72
8       C     eng     60
```

MEDIUM | PIVOT (LONG TO WIDE)

Q65. Convert the long table back into wide format with one row per student.

CONCEPTS USED: pivot(index, columns, values)

SOLUTION

```
long = scores.melt(id_vars='student', var_name='subject', value_name='marks')
print(long.pivot(index='student', columns='subject', values='marks'))
```

OUTPUT

```
subject  eng  math  sci
student
A         88    80   75
B         72    90   85
C         60    70   95
```

MEDIUM | UNSTACK

Q66. Compute the average salary by department and city, then reshape so cities become columns.

CONCEPTS USED: groupby() with two keys, unstack()

SOLUTION

```
res = emp.groupby(['dept', 'city'])['salary'].mean().unstack()
print(res)
```

OUTPUT

```
city       Delhi   Mumbai     Pune
dept
Finance      NaN  61000.0  90000.0
HR       48000.0  75000.0      NaN
IT       69000.0  72000.0  65000.0
Sales    52000.0      NaN  52500.0
```

MEDIUM | ROW-WISE APPLY

Q67. Create a description column for every employee like 'Aarav (IT, Pune)' using a row-wise apply.

CONCEPTS USED: apply(axis=1), f-strings

SOLUTION

```
df = emp.copy()
df['desc'] = df.apply(lambda r: f"{r['name']} ({r['dept']}, {r['city']})", axis=1)
print(df['desc'].head(4).tolist())
```

OUTPUT

```
['Aarav (IT, Pune)', 'Diya (HR, Delhi)', 'Kabir (IT, Mumbai)', 'Meera (Finance, Pune)']
```

MEDIUM | CUSTOM FUNCTIONS

Q68. Write a function that applies tax: 0% up to 50000, 10% on salary above 50000 up to 70000, 20% above 70000 (on the portion above 70000) and apply it to the salary column.

CONCEPTS USED: apply() with a custom function

SOLUTION

```
def tax(s):
    t = 0
    if s > 50000:
        t += (min(s, 70000) - 50000) * 0.10
    if s > 70000:
        t += (s - 70000) * 0.20
    return t

df = emp.copy()
df['tax'] = df['salary'].apply(tax)
print(df[['name', 'salary', 'tax']].head(6))
```

OUTPUT

```
    name  salary     tax
0  Aarav   65000  1500.0
1   Diya   48000     0.0
2  Kabir   72000  2400.0
3  Meera   90000  6000.0
4  Rohan   52000   200.0
5   Sana   61000  1100.0
```

MEDIUM | MULTIPLE CONDITIONS

Q69. Classify salaries into 'High' (>=70000), 'Medium' (50000-69999) and 'Low' (<50000).

CONCEPTS USED: np.select()

SOLUTION

```
df = emp.copy()
conds = [df['salary'] >= 70000, df['salary'] >= 50000]
df['band'] = np.select(conds, ['High', 'Medium'], default='Low')
print(df[['name', 'salary', 'band']])
```

OUTPUT

```
     name  salary    band
0   Aarav   65000  Medium
1    Diya   48000     Low
2   Kabir   72000    High
3   Meera   90000    High
4   Rohan   52000  Medium
5    Sana   61000  Medium
6  Vikram   58000  Medium
7    Isha   69000  Medium
8    Neel   75000    High
9    Tara   47000     Low
```

MEDIUM | QUANTILE BINNING

Q70. Divide employees into 4 equal-sized salary quartiles (Q1 to Q4).

CONCEPTS USED: pd.qcut()

SOLUTION

```
df = emp.copy()
df['quartile'] = pd.qcut(df['salary'], 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
print(df[['name', 'salary', 'quartile']].sort_values('salary'))
```

OUTPUT

```
     name  salary quartile
9    Tara   47000       Q1
1    Diya   48000       Q1
4   Rohan   52000       Q1
6  Vikram   58000       Q2
5    Sana   61000       Q2
0   Aarav   65000       Q3
7    Isha   69000       Q3
2   Kabir   72000       Q4
8    Neel   75000       Q4
3   Meera   90000       Q4
```

MEDIUM | COUNTING BINS

Q71. Count how many employees fall into each salary range: 40k-55k, 55k-70k, 70k-95k.

CONCEPTS USED: pd.cut(), value_counts(sort_index)

SOLUTION

```
b = pd.cut(emp['salary'], bins=[40000, 55000, 70000, 95000])
print(b.value_counts().sort_index())
```

OUTPUT

```
salary
(40000, 55000]    3
(55000, 70000]    4
(70000, 95000]    3
Name: count, dtype: int64
```

MEDIUM | DATETIME COMPONENTS

Q72. From the order_date of sales extract year, month, quarter and day of week (0 = Monday).

CONCEPTS USED: dt.year, dt.month, dt.quarter, dt.dayofweek

SOLUTION

```
d = sales['order_date']
out = pd.DataFrame({'month': d.dt.month, 'quarter': d.dt.quarter, 'dow': d.dt.dayofweek})
print(out.head(5))
```

OUTPUT

```
   month  quarter  dow
0      1        1    4
1      1        1    4
2      1        1    5
3      2        1    5
4      2        1    2
```

MEDIUM | MONTHLY AGGREGATION

Q73. Calculate the total revenue for each month using to_period.

CONCEPTS USED: dt.to_period('M'), groupby()

SOLUTION

```
df = sales.assign(revenue=sales['qty'] * sales['price'])
print(df.groupby(df['order_date'].dt.to_period('M'))['revenue'].sum())
```

OUTPUT

```
order_date
2024-01    121000
2024-02     96160
2024-03    191600
2024-04     25460
Freq: M, Name: revenue, dtype: int64
```

MEDIUM | RESAMPLING

Q74. Use resample to compute the number of orders and total quantity per month.

CONCEPTS USED: set_index(datetime), resample('MS')

SOLUTION

```
df = sales.set_index('order_date')
print(df.resample('MS').agg(orders=('order_id', 'count'), qty=('qty', 'sum')))
```

OUTPUT

```
            orders  qty
order_date
2024-01-01       3   17
2024-02-01       3   12
2024-03-01       3    9
2024-04-01       3   19
```

MEDIUM | DATE DIFFERENCES

Q75. Calculate each employee's tenure in days and in years as of 31 Dec 2024.

CONCEPTS USED: Timestamp subtraction, dt.days

SOLUTION

```
ref = pd.Timestamp('2024-12-31')
df = emp.copy()
df['days'] = (ref - df['join_date']).dt.days
df['years'] = (df['days'] / 365.25).round(1)
print(df[['name', 'days', 'years']].head(5))
```

OUTPUT

```
    name  days  years
0  Aarav  1752    4.8
1   Diya  3105    8.5
2  Kabir  1086    3.0
3  Meera  4974   13.6
4  Rohan  2309    6.3
```

MEDIUM | PARSING DATES

Q76. Convert the text dates ['15/03/2024', '01/12/2023', 'bad'] to datetime (day first), turning invalid ones into NaT.

CONCEPTS USED: pd.to_datetime(format, errors='coerce')

SOLUTION

```
s = pd.Series(['15/03/2024', '01/12/2023', 'bad'])
print(pd.to_datetime(s, format='%d/%m/%Y', errors='coerce'))
```

OUTPUT

```
0   2024-03-15
1   2023-12-01
2          NaT
dtype: datetime64[us]
```

MEDIUM | REGEX EXTRACTION

Q77. From order codes ['ORD-101', 'ORD-205', 'ORD-317'] extract the numeric part as integers.

CONCEPTS USED: str.extract() with regex

SOLUTION

```
codes = pd.Series(['ORD-101', 'ORD-205', 'ORD-317'])
print(codes.str.extract(r'(\d+)')[0].astype(int))
```

OUTPUT

```
0    101
1    205
2    317
Name: 0, dtype: int64
```

MEDIUM | SPLITTING STRINGS

Q78. Split full names ['Ravi Kumar', 'Sita Devi', 'Tom Hardy'] into first and last name columns.

CONCEPTS USED: str.split(expand=True)

SOLUTION

```
names = pd.Series(['Ravi Kumar', 'Sita Devi', 'Tom Hardy'])
out = names.str.split(' ', expand=True)
out.columns = ['first', 'last']
print(out)
```

OUTPUT

```
   first   last
0  Ravi  Kumar
1  Sita   Devi
2   Tom  Hardy
```

MEDIUM | REGEX REPLACE

Q79. Clean phone numbers ['+91-98765 43210', '(022) 2345-6789'] by removing every non-digit character.

CONCEPTS USED: str.replace(regex=True)

SOLUTION

```
ph = pd.Series(['+91-98765 43210', '(022) 2345-6789'])
print(ph.str.replace(r'\D', '', regex=True))
```

OUTPUT

```
0    919876543210
1     02223456789
dtype: str
```

MEDIUM | GROUP-WISE IMPUTATION

Q80. Fill missing salaries with the average salary of the same department.

CONCEPTS USED: groupby(), transform(), fillna()

SOLUTION

```
df = pd.DataFrame({'dept': ['IT', 'IT', 'HR', 'HR', 'IT'],
                   'salary': [60000, np.nan, 50000, 54000, 70000]})
df['salary'] = df['salary'].fillna(df.groupby('dept')['salary'].transform('mean'))
print(df)
```

OUTPUT

```
  dept   salary
0   IT  60000.0
1   IT  65000.0
2   HR  50000.0
3   HR  54000.0
4   IT  70000.0
```

MEDIUM | INTERPOLATION

Q81. A daily temperature Series has missing values. Fill them using linear interpolation.

CONCEPTS USED: interpolate()

SOLUTION

```
s = pd.Series([20.0, np.nan, np.nan, 29.0, 30.0, np.nan, 28.0])
print(s.interpolate().round(2).tolist())
```

OUTPUT

```
[20.0, 23.0, 26.0, 29.0, 30.0, 29.0, 28.0]
```

MEDIUM | FORWARD / BACKWARD FILL

Q82. Fill missing values by carrying the last valid value forward, and then by using the next valid value backward.

CONCEPTS USED: ffill(), bfill()

SOLUTION

```
s = pd.Series([1.0, np.nan, np.nan, 4.0, np.nan])
print(s.ffill().tolist())
print(s.bfill().tolist())
```

OUTPUT

```
[1.0, 1.0, 1.0, 4.0, 4.0]
[1.0, 4.0, 4.0, 4.0, nan]
```

MEDIUM | OUTLIER DETECTION

Q83. Detect outliers in the 'values' list using the IQR rule (1.5 * IQR).

CONCEPTS USED: quantile(), IQR rule

SOLUTION

```
s = pd.Series([12, 14, 13, 15, 14, 13, 95, 12, -30])
q1, q3 = s.quantile(0.25), s.quantile(0.75)
iqr = q3 - q1
mask = (s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)
print(q1, q3, iqr)
print(s[mask].tolist())
```

OUTPUT

```
12.0 14.0 2.0
[95, -30]
```

MEDIUM | CLIPPING

Q84. Limit (winsorize) a Series so that values below 20 become 20 and values above 80 become 80.

CONCEPTS USED: clip()

SOLUTION

```
s = pd.Series([5, 25, 50, 95, 70, 10])
print(s.clip(20, 80).tolist())
```

OUTPUT

```
[20, 25, 50, 80, 70, 20]
```

MEDIUM | REPLACE / WHERE / MASK

Q85. Replace the text 'N/A' with NaN, keep salaries above 55000 using where(), and set values to 0 where salary is below 50000 using mask().

CONCEPTS USED: replace(), where(), mask()

SOLUTION

```
s = pd.Series(['N/A', 'ok', 'N/A'])
print(s.replace('N/A', np.nan).tolist())
sal = emp['salary']
print(sal.where(sal > 55000).head(5).tolist())
print(sal.mask(sal < 50000, 0).tolist())
```

OUTPUT

```
[nan, 'ok', nan]
[65000.0, nan, 72000.0, 90000.0, nan]
[65000, 0, 72000, 90000, 52000, 61000, 58000, 69000, 75000, 0]
```

MEDIUM | CONDITIONAL UPDATE

Q86. Give a 10% raise to all IT employees, in place.

CONCEPTS USED: loc with boolean mask

SOLUTION

```
df = emp.copy()
df.loc[df['dept'] == 'IT', 'salary'] = df.loc[df['dept'] == 'IT', 'salary'] * 1.10
print(df[df['dept'] == 'IT'][['name', 'salary']])
```

OUTPUT

```
    name  salary
0  Aarav   71500
2  Kabir   79200
7   Isha   75900
```

MEDIUM | INDEX OPERATIONS

Q87. Sort a Series by its index and reindex it to include a missing label (filled with 0).

CONCEPTS USED: sort_index(), reindex()

SOLUTION

```
s = pd.Series([3, 1, 2], index=['c', 'a', 'b'])
print(s.sort_index())
print(s.reindex(['a', 'b', 'c', 'd'], fill_value=0))
```

OUTPUT

```
a    1
b    2
c    3
dtype: int64
a    1
b    2
c    3
d    0
dtype: int64
```

MEDIUM | MULTIINDEX

Q88. Set (dept, city) as a MultiIndex and select all rows of the IT department, and then rows from Pune across all departments.

CONCEPTS USED: set_index(list), loc on MultiIndex, xs()

SOLUTION

```
df = emp.set_index(['dept', 'city']).sort_index()
print(df.loc['IT', ['name', 'salary']])
print(df.xs('Pune', level='city')[['name']])
```

OUTPUT

```
          name  salary
city
Delhi     Isha   69000
Mumbai  Kabir   72000
Pune    Aarav   65000
           name
dept
Finance   Meera
IT         Aarav
Sales     Vikram
Sales       Tara
```

MEDIUM | GROUPBY MULTIPLE KEYS

Q89. Find the total quantity and the revenue per region and category and return a flat DataFrame.

CONCEPTS USED: groupby(list), agg(), reset_index()

SOLUTION

```
df = sales.assign(revenue=sales['qty'] * sales['price'])
res = (df.groupby(['region', 'category'])
         .agg(qty=('qty', 'sum'), revenue=('revenue', 'sum'))
         .reset_index())
print(res.head(6))
```

OUTPUT

```
   region     category  qty  revenue
0   East  Accessories   10    12100
1   East     Displays    3    36000
2  North  Accessories   17    11760
3  North    Computers    2   110000
4  North     Displays    2    25000
5  South  Accessories   18     9160
```

MEDIUM | CUMULATIVE IN GROUPS

Q90. For each customer, compute the running count of their orders and a running total of quantity.

CONCEPTS USED: groupby().cumsum(), cumcount()

SOLUTION

```
df = sales[['customer_id', 'order_date', 'qty']].sort_values(['customer_id', 'order_date']).copy()
df['order_no'] = df.groupby('customer_id').cumcount() + 1
df['run_qty'] = df.groupby('customer_id')['qty'].cumsum()
print(df.head(6))
```

OUTPUT

```
   customer_id order_date  qty  order_no  run_qty
0           C1 2024-01-05    2         1        2
2           C1 2024-01-20    5         2        7
6           C1 2024-03-02    2         3        9
1           C2 2024-01-12   10         1       10
4           C2 2024-02-14    1         2       11
10          C2 2024-04-18    1         3       12
```

MEDIUM | DIFFERENCES & CHANGE

Q91. For the ts price series, compute the daily change and daily percentage change (first 5 rows).

CONCEPTS USED: diff(), pct_change()

SOLUTION

```
out = ts[['date', 'price']].copy()
out['chg'] = out['price'].diff().round(2)
out['pct'] = (out['price'].pct_change() * 100).round(2)
print(out.head(5))
```

OUTPUT

```
        date   price   chg   pct
0 2024-01-01  100.25   NaN   NaN
1 2024-01-02   99.99 -0.26 -0.26
2 2024-01-03  101.27  1.28  1.28
3 2024-01-04  101.48  0.21  0.21
4 2024-01-05  100.41 -1.07 -1.05
```

MEDIUM | ROLLING WINDOW

Q92. Calculate a 5-day moving average of price.

CONCEPTS USED: rolling(window).mean()

SOLUTION

```
out = ts[['date', 'price']].copy()
out['ma5'] = out['price'].rolling(5).mean().round(2)
print(out.iloc[3:9])
```

OUTPUT

```
        date   price     ma5
3 2024-01-04  101.48     NaN
4 2024-01-05  100.41  100.68
5 2024-01-06  101.13  100.86
6 2024-01-07  103.74  101.61
7 2024-01-08  105.63  102.48
8 2024-01-09  104.22  103.03
```

MEDIUM | EXPANDING WINDOW

Q93. Find the highest price seen so far (running maximum) for every day.

CONCEPTS USED: expanding().max(), cummax()

SOLUTION

```
out = ts[['date', 'price']].copy()
out['run_max'] = out['price'].expanding().max()
print(out.head(6))
print((out['run_max'] == out['price'].cummax()).all())
```

OUTPUT

```
        date   price  run_max
0 2024-01-01  100.25   100.25
1 2024-01-02   99.99   100.25
2 2024-01-03  101.27   101.27
3 2024-01-04  101.48   101.48
4 2024-01-05  100.41   101.48
5 2024-01-06  101.13   101.48
True
```

MEDIUM | SHIFTING

Q94. Add the previous day's price and the next day's price as new columns, and check if the price went up compared to yesterday.

CONCEPTS USED: shift(), shift(-1)

SOLUTION

```
out = ts[['date', 'price']].copy()
out['prev'] = out['price'].shift(1)
out['next'] = out['price'].shift(-1)
out['up'] = out['price'] > out['prev']
print(out.head(5))
```

OUTPUT

```
        date   price    prev    next     up
0 2024-01-01  100.25     NaN   99.99  False
1 2024-01-02   99.99  100.25  101.27  False
2 2024-01-03  101.27   99.99  101.48   True
3 2024-01-04  101.48  101.27  100.41   True
4 2024-01-05  100.41  101.48  101.13  False
```

MEDIUM | SIZE VS COUNT

Q95. Show the difference between size() and count() in a group-by with missing values, and find the number of unique products per region.

CONCEPTS USED: groupby().size(), count(), nunique()

SOLUTION

```
df = pd.DataFrame({'g': ['a', 'a', 'b'], 'v': [1, np.nan, 3]})
print(df.groupby('g').size().tolist(), df.groupby('g')['v'].count().tolist())
print(sales.groupby('region')['product'].nunique())
```

OUTPUT

```
[2, 1] [1, 1]
region
East     2
North    4
South    2
West     1
Name: product, dtype: int64
```

MEDIUM | TOP-N PER GROUP

Q96. Find the top 2 highest-paid employees in each department.

CONCEPTS USED: sort_values(), groupby().head(n)

SOLUTION

```
res = emp.sort_values('salary', ascending=False).groupby('dept').head(2)
print(res.sort_values(['dept', 'salary'], ascending=[True, False])[['dept', 'name', 'salary']])
```

OUTPUT

```
      dept    name  salary
3  Finance   Meera   90000
5  Finance    Sana   61000
8       HR    Neel   75000
1       HR    Diya   48000
2       IT   Kabir   72000
7       IT    Isha   69000
6    Sales  Vikram   58000
4    Sales   Rohan   52000
```

MEDIUM | PARETO ANALYSIS

Q97. Show the share and the cumulative share of total revenue contributed by each product (Pareto analysis).

CONCEPTS USED: value_counts(), cumsum(), normalize

SOLUTION

```
df = sales.assign(revenue=sales['qty'] * sales['price'])
r = df.groupby('product')['revenue'].sum().sort_values(ascending=False)
out = pd.DataFrame({'share_%': (r / r.sum() * 100).round(1)})
out['cum_%'] = out['share_%'].cumsum().round(1)
print(out)
```

OUTPUT

```
           share_%  cum_%
product
Laptop        75.5   75.5
Monitor       16.9   92.4
Keyboard       4.2   96.6
Mouse          3.4  100.0
```

MEDIUM | ONE-HOT ENCODING

Q98. One-hot encode the 'dept' column of emp into dummy columns (0/1).

CONCEPTS USED: pd.get_dummies()

SOLUTION

```
d = pd.get_dummies(emp['dept'], prefix='dept', dtype=int)
print(pd.concat([emp['name'], d], axis=1).head(5))
```

OUTPUT

```
     name  dept_Finance  dept_HR  dept_IT  dept_Sales
0  Aarav             0        0        1           0
1   Diya             0        1        0           0
2  Kabir             0        0        1           0
3  Meera             1        0        0           0
4  Rohan             0        0        0           1
```

MEDIUM | CATEGORY DTYPE

Q99. Convert the 'city' column to the category dtype and compare memory usage before and after on a large repeated column.

CONCEPTS USED: astype('category'), memory_usage()

SOLUTION

```
big = pd.DataFrame({'city': ['Pune', 'Delhi', 'Mumbai'] * 10000})
before = big.memory_usage(deep=True).sum()
big['city'] = big['city'].astype('category')
after = big.memory_usage(deep=True).sum()
print(before > after, big['city'].cat.categories.tolist())
```

OUTPUT

```
True ['Delhi', 'Mumbai', 'Pune']
```

MEDIUM | PERCENT OF GROUP TOTAL

Q100. Show each employee's salary as a percentage of their department's total salary.

CONCEPTS USED: groupby().transform('sum'), division

SOLUTION

```
df = emp.copy()
df['pct_of_dept'] = (df['salary'] / df.groupby('dept')['salary'].transform('sum') * 100).round(1)
print(df[['dept', 'name', 'salary', 'pct_of_dept']].sort_values('dept'))
```

OUTPUT

```
      dept    name  salary  pct_of_dept
3  Finance   Meera   90000         59.6
5  Finance    Sana   61000         40.4
1       HR    Diya   48000         39.0
8       HR    Neel   75000         61.0
0       IT   Aarav   65000         31.6
2       IT   Kabir   72000         35.0
7       IT    Isha   69000         33.5
4    Sales   Rohan   52000         33.1
6    Sales  Vikram   58000         36.9
9    Sales    Tara   47000         29.9
```

SECTION C - ADVANCED LEVEL

Q101 to Q150 | Window functions, cohorts, time series, pipelines, mini projects

ADVANCED | ROLLING STATISTICS

Q101. Compute a 7-day rolling standard deviation of price (allow partial windows with min_periods=3) and show days 1 to 6.

CONCEPTS USED: rolling(window, min_periods), std()

SOLUTION

```
r = ts['price'].rolling(7, min_periods=3).std().round(3)
print(r.iloc[:6].tolist())
```

OUTPUT

```
[nan, nan, 0.677, 0.737, 0.656, 0.615]
```

ADVANCED | EXPONENTIAL WEIGHTED MEAN

Q102. Calculate a 10-day exponentially weighted moving average of price and compare it with the simple 10-day average (last 3 rows).

CONCEPTS USED: ewm(span).mean()

SOLUTION

```
out = pd.DataFrame({'sma10': ts['price'].rolling(10).mean(),
                    'ema10': ts['price'].ewm(span=10, adjust=False).mean()}).round(2)
print(out.tail(3))
```

OUTPUT

```
     sma10  ema10
27   94.02  94.53
28   94.19  94.12
29   94.20  93.86
```

ADVANCED | ROLLING INSIDE GROUPS

Q103. For every customer compute the rolling sum of the last 2 orders' quantity.

CONCEPTS USED: groupby().rolling(), reset_index(level)

SOLUTION

```
df = sales.sort_values(['customer_id', 'order_date'])
r = df.groupby('customer_id')['qty'].rolling(2).sum()
print(r.reset_index(level=0, drop=True).sort_index().head(8).tolist())
```

OUTPUT

```
[nan, nan, 7.0, nan, 11.0, nan, 7.0, 7.0]
```

ADVANCED | PERCENTILE RANK

Q104. Compute the percentile rank of every employee's salary within the whole company and within their department.

CONCEPTS USED: rank(pct=True)

SOLUTION

```
df = emp.copy()
df['pct_all'] = df['salary'].rank(pct=True).round(2)
df['pct_dept'] = df.groupby('dept')['salary'].rank(pct=True).round(2)
print(df[['name', 'dept', 'pct_all', 'pct_dept']].head(6))
```

OUTPUT

```
    name     dept  pct_all  pct_dept
0  Aarav       IT      0.6      0.33
1   Diya       HR      0.2      0.50
2  Kabir       IT      0.8      1.00
3  Meera  Finance      1.0      1.00
4  Rohan    Sales      0.3      0.67
5   Sana  Finance      0.5      0.50
```

ADVANCED | GROUPBY APPLY (DATAFRAME)

Q105. For each department return a custom summary: salary range (max - min) and the name of the most experienced employee.

CONCEPTS USED: groupby().apply(), returning a Series

SOLUTION

```
def summ(g):
    return pd.Series({'salary_range': g['salary'].max() - g['salary'].min(),
                      'most_exp': g.loc[g['exp'].idxmax(), 'name']})
print(emp.groupby('dept')[['salary', 'exp', 'name']].apply(summ))
```

OUTPUT

```
         salary_range most_exp
dept
Finance         29000    Meera
HR              27000     Neel
IT               7000    Aarav
Sales           11000   Vikram
```

ADVANCED | Z-SCORE WITHIN GROUPS

Q106. Standardize salary within each department (z-score) and flag employees with |z| > 1.

CONCEPTS USED: groupby().transform(lambda), std()

SOLUTION

```
df = emp.copy()
g = df.groupby('dept')['salary']
df['z'] = ((df['salary'] - g.transform('mean')) / g.transform('std')).round(2)
df['flag'] = df['z'].abs() > 1
print(df[['dept', 'name', 'z', 'flag']].sort_values('dept'))
```

OUTPUT

```
      dept    name     z   flag
3  Finance   Meera  0.71  False
5  Finance    Sana -0.71  False
1       HR    Diya -0.71  False
8       HR    Neel  0.71  False
0       IT   Aarav -1.04   True
2       IT   Kabir  0.95  False
7       IT    Isha  0.09  False
4    Sales   Rohan -0.06  False
6    Sales  Vikram  1.03   True
9    Sales    Tara -0.97  False
```

ADVANCED | MULTI-AGGREGATION PIVOT

Q107. Create a pivot table with the sum and mean of revenue for each category by region, filling missing cells with 0.

CONCEPTS USED: pivot_table() with multiple aggfunc and values

SOLUTION

```
df = sales.assign(revenue=sales['qty'] * sales['price'])
pt = df.pivot_table(index='category', columns='region', values='revenue',
                    aggfunc=['sum', 'mean'], fill_value=0).round(0)
print(pt['sum'])
```

OUTPUT

```
region        East   North  South    West
category
Accessories  12100   11760   9160       0
Computers        0  110000      0  218000
Displays     36000   25000  12200       0
```

ADVANCED | FLATTEN MULTIINDEX COLUMNS

Q108. After a groupby with several aggregations the columns become a MultiIndex. Flatten them into names like 'salary_mean'.

CONCEPTS USED: MultiIndex columns, join of tuples

SOLUTION

```
res = emp.groupby('dept').agg({'salary': ['mean', 'max'], 'exp': ['mean']})
res.columns = ['_'.join(c) for c in res.columns]
print(res.round(1))
```

OUTPUT

```
         salary_mean  salary_max  exp_mean
dept
Finance      75500.0       90000      10.0
HR           61500.0       75000      13.5
IT           68666.7       72000       3.0
Sales        52333.3       58000       8.3
```

ADVANCED | NORMALIZED CROSSTAB

Q109. Show, for each department, the percentage of employees located in each city (rows add up to 100%).

CONCEPTS USED: crosstab(normalize='index')

SOLUTION

```
ct = pd.crosstab(emp['dept'], emp['city'], normalize='index') * 100
print(ct.round(0))
```

OUTPUT

```
city     Delhi  Mumbai  Pune
dept
Finance    0.0    50.0  50.0
HR        50.0    50.0   0.0
IT        33.0    33.0  33.0
Sales     33.0     0.0  67.0
```

ADVANCED | MERGE_ASOF

Q110. Match each trade with the most recent quote available at or before the trade time.

CONCEPTS USED: merge_asof(), sort by key, direction

SOLUTION

```
quotes = pd.DataFrame({'time': pd.to_datetime(['2024-01-01 10:00:00',
                                               '2024-01-01 10:00:05',
                                               '2024-01-01 10:00:10']),
                       'bid': [100.0, 100.5, 101.0]})
trades = pd.DataFrame({'time': pd.to_datetime(['2024-01-01 10:00:03',
                                               '2024-01-01 10:00:07',
                                               '2024-01-01 10:00:12']),
                       'qty': [10, 20, 30]})
print(pd.merge_asof(trades, quotes, on='time'))
```

OUTPUT

```
                 time  qty    bid
0 2024-01-01 10:00:03   10  100.0
1 2024-01-01 10:00:07   20  100.5
2 2024-01-01 10:00:12   30  101.0
```

ADVANCED | MERGE VALIDATION

Q111. Merge emp and cust on city using suffixes to avoid column clashes, and use validate to catch unexpected many-to-many joins.

CONCEPTS USED: merge(suffixes, validate)

SOLUTION

```
a = emp[['name', 'city']].head(3)
b = cust[['cust_name', 'city']].drop_duplicates('city')
m = a.merge(b, on='city', how='left', validate='many_to_one')
print(m)
try:
    emp.merge(cust, on='city', validate='one_to_one')
except Exception as e:
    print('Error:', type(e).__name__)
```

OUTPUT

```
    name    city cust_name
0  Aarav    Pune     Rahul
1   Diya   Delhi     Anita
2  Kabir  Mumbai    Suresh
Error: MergeError
```

ADVANCED | CONCAT WITH KEYS

Q112. Concatenate two DataFrames with keys to identify their origin, then fill the gaps of one with values of another using combine_first.

CONCEPTS USED: concat(keys=), combine_first()

SOLUTION

```
a = pd.DataFrame({'x': [1, np.nan]}, index=['r1', 'r2'])
b = pd.DataFrame({'x': [9, 2]}, index=['r1', 'r2'])
print(pd.concat([a, b], keys=['A', 'B']))
print(a.combine_first(b))
```

OUTPUT

```
        x
A r1  1.0
  r2  NaN
B r1  9.0
  r2  2.0
     x
r1  1.0
r2  2.0
```

ADVANCED | EXPLODE

Q113. A column holds lists of skills. Convert it to one row per skill and count how many people have each skill.

CONCEPTS USED: explode(), split + explode

SOLUTION

```
df = pd.DataFrame({'name': ['A', 'B'], 'skills': [['py', 'sql'], ['py', 'ml', 'sql']]})
ex = df.explode('skills')
print(ex)
print(ex['skills'].value_counts())
```

OUTPUT

```
  name skills
0    A     py
0    A    sql
1    B     py
1    B     ml
1    B    sql
skills
py     2
sql    2
ml     1
Name: count, dtype: int64
```

ADVANCED | NESTED JSON

Q114. Flatten nested JSON-like records (user with address) into a flat table.

CONCEPTS USED: pd.json_normalize()

SOLUTION

```
data = [{'id': 1, 'user': {'name': 'Ravi', 'city': 'Pune'}, 'amt': 250},
        {'id': 2, 'user': {'name': 'Sita', 'city': 'Delhi'}, 'amt': 400}]
print(pd.json_normalize(data))
```

OUTPUT

```
   id  amt user.name user.city
0   1  250      Ravi      Pune
1   2  400      Sita     Delhi
```

ADVANCED | NAMED REGEX GROUPS

Q115. Extract the year, month and day from strings like '2024-03-15' into separate integer columns using named groups.

CONCEPTS USED: str.extract() with named groups

SOLUTION

```
s = pd.Series(['2024-03-15', '2023-11-02'])
out = s.str.extract(r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})').astype(int)
print(out)
```

OUTPUT

```
   year  month  day
0  2024      3   15
1  2023     11    2
```

ADVANCED | VECTORIZE VS APPLY

Q116. Rewrite a slow row-wise apply that labels employees as 'High' (salary > 60000) or 'Low' using a vectorized approach and prove both give the same result.

CONCEPTS USED: np.where / vectorized ops vs apply, equals()

SOLUTION

```
slow = emp.apply(lambda r: 'High' if r['salary'] > 60000 else 'Low', axis=1)
fast = pd.Series(np.where(emp['salary'] > 60000, 'High', 'Low'), index=emp.index)
print(slow.tolist() == fast.tolist())
print(fast.value_counts().to_dict())
```

OUTPUT

```
True
{'High': 6, 'Low': 4}
```

ADVANCED | DRAWDOWN ANALYSIS

Q117. Compute the running drawdown of the price series and report the maximum drawdown (in %).

CONCEPTS USED: cummax(), division, min()

SOLUTION

```
p = ts['price']
dd = p / p.cummax() - 1
print(round(dd.min() * 100, 2))
print(ts.loc[dd.idxmin(), 'date'].date())
```

OUTPUT

```
-15.02
2024-01-18
```

ADVANCED | OHLC RESAMPLING

Q118. Convert the daily price series into weekly Open-High-Low-Close data.

CONCEPTS USED: set_index(), resample('W').ohlc()

SOLUTION

```
w = ts.set_index('date')['price'].resample('W').ohlc()
print(w.head(3))
```

OUTPUT

```
              open    high    low   close
date
2024-01-07  100.25  103.74  99.99  103.74
2024-01-14  105.63  105.63  95.44   95.44
2024-01-21   92.95   92.95  89.76   92.42
```

ADVANCED | BUSINESS DAYS

Q119. Generate all business days (Mon-Fri) in January 2024 and count how many there are. Also generate month-end dates for 2024 Q1.

CONCEPTS USED: pd.bdate_range(), date_range(freq)

SOLUTION

```
b = pd.bdate_range('2024-01-01', '2024-01-31')
print(len(b), b[0].date(), b[-1].date())
print(pd.date_range('2024-01-01', periods=3, freq='ME').strftime('%Y-%m-%d').tolist())
```

OUTPUT

```
23 2024-01-01 2024-01-31
['2024-01-31', '2024-02-29', '2024-03-31']
```

ADVANCED | TIME ZONES

Q120. Convert a naive timestamp assumed in UTC to Indian Standard Time (Asia/Kolkata) and to New York time.

CONCEPTS USED: tz_localize(), tz_convert()

SOLUTION

```
t = pd.Series(pd.to_datetime(['2024-03-15 12:00']))
utc = t.dt.tz_localize('UTC')
print(utc.dt.tz_convert('Asia/Kolkata').iloc[0])
print(utc.dt.tz_convert('America/New_York').iloc[0])
```

OUTPUT

```
2024-03-15 17:30:00+05:30
2024-03-15 08:00:00-04:00
```

ADVANCED | PERIODS & OFFSETS

Q121. Find the last day of the month for each order date and the first day of its next month using offsets.

CONCEPTS USED: to_period(), pd.offsets

SOLUTION

```
d = sales['order_date'].head(3)
print((d + pd.offsets.MonthEnd(0)).dt.date.tolist())
print((d + pd.offsets.MonthBegin(1)).dt.date.tolist())
```

OUTPUT

```
[datetime.date(2024, 1, 31), datetime.date(2024, 1, 31), datetime.date(2024, 1, 31)]
[datetime.date(2024, 2, 1), datetime.date(2024, 2, 1), datetime.date(2024, 2, 1)]
```

ADVANCED | ROLLING CORRELATION

Q122. Compute the 10-day rolling correlation between price and volume and show the last value.

CONCEPTS USED: rolling().corr()

SOLUTION

```
rc = ts['price'].rolling(10).corr(ts['volume'])
print(round(rc.iloc[-1], 4))
print(rc.notna().sum())
```

OUTPUT

```
0.1234
21
```

ADVANCED | COHORT ANALYSIS

Q123. For each order, find the customer's first-order month (cohort) and how many months after that the order happened.

CONCEPTS USED: groupby().transform('min'), to_period, month difference

SOLUTION

```
df = sales[['customer_id', 'order_date']].copy()
df['month'] = df['order_date'].dt.to_period('M')
df['cohort'] = df.groupby('customer_id')['month'].transform('min')
df['age'] = (df['month'] - df['cohort']).apply(lambda x: x.n)
print(df.sort_values(['customer_id', 'order_date']).head(6))
```

OUTPUT

```
   customer_id order_date    month   cohort  age
0           C1 2024-01-05  2024-01  2024-01    0
2           C1 2024-01-20  2024-01  2024-01    0
6           C1 2024-03-02  2024-03  2024-01    2
1           C2 2024-01-12  2024-01  2024-01    0
4           C2 2024-02-14  2024-02  2024-01    1
10          C2 2024-04-18  2024-04  2024-01    3
```

ADVANCED | RFM ANALYSIS

Q124. Create an RFM table (Recency in days, Frequency, Monetary) for each customer as of 2024-05-01 and score Monetary into 3 groups.

CONCEPTS USED: groupby().agg(), rank(), qcut

SOLUTION

```
df = sales.assign(rev=sales['qty'] * sales['price'])
rfm = df.groupby('customer_id').agg(last=('order_date', 'max'),
                                    freq=('order_id', 'count'),
                                    monetary=('rev', 'sum'))
rfm['recency'] = (pd.Timestamp('2024-05-01') - rfm['last']).dt.days
rfm['m_score'] = pd.qcut(rfm['monetary'].rank(method='first'), 3, labels=[1, 2, 3])
print(rfm[['recency', 'freq', 'monetary', 'm_score']])
```

OUTPUT

```
             recency  freq  monetary m_score
customer_id
C1                60     3    141000       3
C2                13     3     73200       2
C3                47     2     40600       1
C4                27     2      9920       1
C5                 1     2    169500       3
```

ADVANCED | REPEAT CUSTOMERS

Q125. Flag every order as coming from a 'repeat' customer (customer has more than 1 order) and compute the share of repeat revenue.

CONCEPTS USED: groupby().size(), map(), boolean flag

SOLUTION

```
df = sales.assign(rev=sales['qty'] * sales['price'])
n = df.groupby('customer_id')['order_id'].transform('size')
df['repeat'] = n > 1
print(df.groupby('repeat')['rev'].sum())
print(round(df.loc[df['repeat'], 'rev'].sum() / df['rev'].sum(), 3))
```

OUTPUT

```
repeat
True    434220
Name: rev, dtype: int64
1.0
```

ADVANCED | RUNNING SHARE

Q126. Within each region show the running revenue total and the order's share of that region's total revenue.

CONCEPTS USED: groupby().cumsum(), transform('sum')

SOLUTION

```
df = sales.assign(rev=sales['qty'] * sales['price']).sort_values(['region', 'order_date'])
df['run'] = df.groupby('region')['rev'].cumsum()
df['share'] = (df['rev'] / df.groupby('region')['rev'].transform('sum')).round(2)
print(df[['region', 'order_id', 'rev', 'run', 'share']].head(6))
```

OUTPUT

```
    region  order_id     rev     run  share
3     East         4   36000   36000   0.75
7     East         8    4600   40600   0.10
11    East        12    7500   48100   0.16
0    North         1  110000  110000   0.75
2    North         3    6000  116000   0.04
6    North         7   25000  141000   0.17
```

ADVANCED | CONSECUTIVE STREAKS

Q127. Find the longest streak of consecutive days on which the price went up.

CONCEPTS USED: diff(), cumsum() grouping trick, groupby().size()

SOLUTION

```
up = ts['price'].diff() > 0
grp = (up != up.shift()).cumsum()
streak = up.groupby(grp).sum()
print(int(streak.max()))
```

OUTPUT

```
3
```

ADVANCED | SESSIONIZATION

Q128. Assign session IDs to click events: a new session starts when the gap from the previous click exceeds 30 minutes.

CONCEPTS USED: diff(), Timedelta comparison, cumsum()

SOLUTION

```
times = ['09:00', '09:10', '09:25', '10:30', '10:40', '12:00']
ev = pd.Series(pd.to_datetime(['2024-01-01 ' + t for t in times]))
sid = (ev.diff() > pd.Timedelta('30min')).cumsum() + 1
print(sid.tolist())
```

OUTPUT

```
[1, 1, 1, 2, 2, 3]
```

ADVANCED | FILLING MISSING DATES

Q129. A series has missing days. Reindex it to a complete daily range and fill the gaps with 0.

CONCEPTS USED: asfreq(), reindex(date_range), fillna

SOLUTION

```
s = pd.Series([5, 7, 9], index=pd.to_datetime(['2024-01-01', '2024-01-03', '2024-01-06']))
full = s.reindex(pd.date_range('2024-01-01', '2024-01-06'), fill_value=0)
print(full)
```

OUTPUT

```
2024-01-01    5
2024-01-02    0
2024-01-03    7
2024-01-04    0
2024-01-05    0
2024-01-06    9
Freq: D, dtype: int64
```

ADVANCED | ROLLING Z-SCORE ANOMALIES

Q130. Flag days when volume is more than 1.5 standard deviations away from its 10-day rolling average.

CONCEPTS USED: rolling mean/std, boolean flag

SOLUTION

```
v = ts['volume']
z = (v - v.rolling(10).mean()) / v.rolling(10).std()
print(ts.loc[z.abs() > 1.5, ['date', 'volume']])
print(z.abs().max().round(2))
```

OUTPUT

```
         date  volume
15 2024-01-16    4557
16 2024-01-17    1291
1.8
```

ADVANCED | INTERVAL BINNING

Q131. Bin experience into custom intervals (0-5, 5-10, 10-20) and compute the average salary in every interval.

CONCEPTS USED: pd.IntervalIndex, cut(), groupby(observed=)

SOLUTION

```
b = pd.cut(emp['exp'], pd.IntervalIndex.from_breaks([0, 5, 10, 20]))
print(emp.groupby(b, observed=True)['salary'].mean().round(0))
```

OUTPUT

```
exp
(0, 5]      66750.0
(5, 10]     49000.0
(10, 20]    74333.0
Name: salary, dtype: float64
```

ADVANCED | ORDERED CATEGORICAL

Q132. Create an ordered categorical for sizes (S < M < L < XL), sort by it and compare using >.

CONCEPTS USED: CategoricalDtype(ordered=True), sort

SOLUTION

```
from pandas.api.types import CategoricalDtype
t = CategoricalDtype(['S', 'M', 'L', 'XL'], ordered=True)
s = pd.Series(['L', 'S', 'XL', 'M', 'S']).astype(t)
print(s.sort_values().tolist())
print((s > 'M').tolist())
```

OUTPUT

```
['S', 'S', 'M', 'L', 'XL']
[True, False, True, False, False]
```

ADVANCED | DOWNCASTING

Q133. Reduce memory by downcasting numeric columns of a large DataFrame and show the saving.

CONCEPTS USED: pd.to_numeric(downcast), memory_usage()

SOLUTION

```
rng2 = np.random.default_rng(0)
df = pd.DataFrame({'a': np.arange(100000), 'b': rng2.integers(0, 100, 100000)})
before = df.memory_usage().sum()
for c in df:
    df[c] = pd.to_numeric(df[c], downcast='unsigned')
print(df.dtypes.astype(str).tolist())
print(before, df.memory_usage().sum())
```

OUTPUT

```
['uint32', 'uint8']
1600132 500132
```

ADVANCED | QUERY & EVAL

Q134. Use query with an external variable and eval to create a new column in one expression.

CONCEPTS USED: query(@variable), eval()

SOLUTION

```
limit = 60000
print(emp.query('salary > @limit and exp < 10')['name'].tolist())
df = emp.copy()
df['salary_per_age'] = df.eval('salary / age')
print(df['salary_per_age'].round(0).head(3).tolist())
```

OUTPUT

```
['Aarav', 'Kabir', 'Sana', 'Isha']
[2321.0, 1412.0, 2880.0]
```

ADVANCED | PIPE()

Q135. Build a clean method chain using assign and pipe that adds revenue, filters revenue > 20000 and returns the top 3 orders.

CONCEPTS USED: pipe(), assign(), method chaining

SOLUTION

```
def top_n(d, n):
    return d.nlargest(n, 'revenue')

res = (sales.assign(revenue=lambda d: d['qty'] * d['price'])
            .query('revenue > 20000')
            .pipe(top_n, 3)
            [['order_id', 'product', 'revenue']])
print(res)
```

OUTPUT

```
   order_id product  revenue
8         9  Laptop   162000
0         1  Laptop   110000
4         5  Laptop    56000
```

ADVANCED | FULL METHOD CHAIN

Q136. In one chain, compute the revenue per month and region, then show the top region of every month.

CONCEPTS USED: assign, groupby, agg, sort_values, reset_index

SOLUTION

```
res = (sales.assign(rev=sales['qty'] * sales['price'],
                    month=sales['order_date'].dt.strftime('%Y-%m'))
        .groupby(['month', 'region'], as_index=False)['rev'].sum()
        .sort_values(['month', 'rev'], ascending=[True, False])
        .groupby('month').head(1))
print(res)
```

OUTPUT

```
      month region     rev
0   2024-01  North  116000
4   2024-02   West   56000
7   2024-03   West  162000
10  2024-04  South   12200
```

ADVANCED | CUSTOM AGGREGATIONS

Q137. For each department, compute the salary range, the most common city and the number of unique cities in one aggregation.

CONCEPTS USED: agg() with lambda, mode(), nunique

SOLUTION

```
res = emp.groupby('dept').agg(
    sal_range=('salary', lambda s: s.max() - s.min()),
    top_city=('city', lambda s: s.mode().iloc[0]),
    n_cities=('city', 'nunique'))
print(res)
```

OUTPUT

```
         sal_range top_city  n_cities
dept
Finance      29000   Mumbai         2
HR           27000    Delhi         2
IT            7000    Delhi         3
Sales        11000     Pune         2
```

ADVANCED | WEIGHTED AVERAGE

Q138. Compute the quantity-weighted average price for each product (sum of qty*price divided by sum of qty).

CONCEPTS USED: groupby, numpy average / vectorized weights

SOLUTION

```
df = sales.assign(w=sales['qty'] * sales['price'])
g = df.groupby('product')
print((g['w'].sum() / g['qty'].sum()).round(1))
```

OUTPUT

```
product
Keyboard     1206.7
Laptop      54666.7
Monitor     12200.0
Mouse         497.3
dtype: float64
```

ADVANCED | TOP-K PER GROUP (NLARGEST)

Q139. Get the 2 most expensive orders (by revenue) for every region.

CONCEPTS USED: groupby().apply(nlargest) / sort + head

SOLUTION

```
df = sales.assign(rev=sales['qty'] * sales['price'])
res = df.sort_values('rev', ascending=False).groupby('region').head(2)
res = res.sort_values(['region', 'rev'], ascending=[True, False])
print(res[['region', 'order_id', 'rev']].head(6))
```

OUTPUT

```
    region  order_id     rev
3    East          4   36000
11   East         12    7500
0   North          1  110000
6   North          7   25000
10  South         11   12200
1   South          2    5000
```

ADVANCED | KEEP BEST RECORD

Q140. For duplicate ids in dup, keep the record with the highest 'val' for each id.

CONCEPTS USED: sort_values() + drop_duplicates(keep='first')

SOLUTION

```
res = dup.sort_values('val', ascending=False).drop_duplicates('id').sort_values('id')
print(res)
```

OUTPUT

```
    id  val
0   1   10
1   2   20
4   3   35
```

ADVANCED | CROSS JOIN

Q141. Generate all (product, region) combinations using a cross join and find the combinations that never had a sale.

CONCEPTS USED: merge(how='cross'), filtering

SOLUTION

```
combos = pd.DataFrame({'product': sales['product'].unique()}).merge(
          pd.DataFrame({'region': sales['region'].unique()}), how='cross')
seen = sales[['product', 'region']].drop_duplicates()
m = combos.merge(seen, how='left', indicator=True)
print(len(combos), (m['_merge'] == 'left_only').sum())
```

OUTPUT

```
16 7
```

ADVANCED | SELF JOIN

Q142. Find pairs of employees working in the same department where the first earns at least 15000 more than the second.

CONCEPTS USED: merge() with itself, suffixes, filtering

SOLUTION

```
a = emp[['name', 'dept', 'salary']]
m = a.merge(a, on='dept', suffixes=('_hi', '_lo'))
res = m[(m['salary_hi'] - m['salary_lo']) >= 15000]
print(res[['dept', 'name_hi', 'name_lo']])
```

OUTPUT

```
        dept name_hi name_lo
9    Finance   Meera    Sana
21       HR    Neel    Diya
```

ADVANCED | MONTH-OVER-MONTH GROWTH

Q143. Compute the month-over-month revenue growth (%) for every category.

CONCEPTS USED: groupby, pct_change(), unstack()

SOLUTION

```
df = sales.assign(rev=sales['qty'] * sales['price'])
m = df.groupby([df['order_date'].dt.to_period('M'), 'category'])['rev'].sum().unstack()
print((m.pct_change() * 100).round(1))
```

OUTPUT

```
category    Accessories  Computers  Displays
order_date
2024-01             NaN        NaN       NaN
2024-02           -62.2      -49.1       NaN
2024-03            10.6      189.3     -30.6
2024-04           188.3        NaN     -51.2
```

ADVANCED | TOP CORRELATED PAIRS

Q144. From the correlation matrix of age, salary, exp and join year, list unique pairs sorted by absolute correlation.

CONCEPTS USED: corr(), unstack(), sort_values, mask

SOLUTION

```
df = emp[['age', 'salary', 'exp']].assign(join_year=emp['join_date'].dt.year)
c = df.corr()
pairs = c.where(np.triu(np.ones(c.shape), k=1).astype(bool)).stack()
print(pairs.reindex(pairs.abs().sort_values(ascending=False).index).round(2).head(4))
```

OUTPUT

```
age     exp          1.00
exp     join_year   -1.00
age     join_year   -1.00
salary  exp          0.35
dtype: float64
```

ADVANCED | IMPUTATION WITH FLAG

Q145. Impute missing salary values with the department median and add a flag column showing which values were imputed.

CONCEPTS USED: groupby median, fillna, isna() indicator

SOLUTION

```
df = pd.DataFrame({'dept': ['A', 'A', 'A', 'B', 'B'], 'sal': [10., np.nan, 14., 20., np.nan]})
df['was_missing'] = df['sal'].isna()
df['sal'] = df['sal'].fillna(df.groupby('dept')['sal'].transform('median'))
print(df)
```

OUTPUT

```
  dept   sal  was_missing
0    A  10.0        False
1    A  12.0         True
2    A  14.0        False
3    B  20.0        False
4    B  20.0         True
```

ADVANCED | STRATIFIED SAMPLING

Q146. Take a stratified sample of 1 employee from every department (reproducible).

CONCEPTS USED: groupby().sample(), random_state

SOLUTION

```
res = emp.groupby('dept', group_keys=False).sample(1, random_state=0)
print(res[['dept', 'name']].sort_values('dept'))
```

OUTPUT

```
      dept   name
5  Finance   Sana
1       HR   Diya
7       IT   Isha
4    Sales  Rohan
```

ADVANCED | TRAIN-TEST SPLIT

Q147. Split emp into 70% training and 30% test sets without using sklearn, and check that they do not overlap.

CONCEPTS USED: sample(frac), drop(index)

SOLUTION

```
train = emp.sample(frac=0.7, random_state=42)
test = emp.drop(train.index)
print(len(train), len(test), set(train.index).isdisjoint(test.index))
```

OUTPUT

```
7 3 True
```

ADVANCED | FEATURE ENGINEERING

Q148. Build a model-ready feature table from emp: one-hot encoded dept, min-max scaled salary and an age-group flag.

CONCEPTS USED: get_dummies, min-max scaling, cut, concat

SOLUTION

```
feat = pd.DataFrame({'sal_scaled': ((emp['salary'] - emp['salary'].min()) /
                                    (emp['salary'].max() - emp['salary'].min())).round(2),
                     'is_senior': (emp['age'] >= 35).astype(int)})
feat = pd.concat([feat, pd.get_dummies(emp['dept'], prefix='d', dtype=int)], axis=1)
print(feat.head(4))
```

OUTPUT

```
   sal_scaled  is_senior  d_Finance  d_HR  d_IT  d_Sales
0        0.42          0          0     0     1        0
1        0.02          0          0     1     0        0
2        0.58          0          0     0     1        0
3        1.00          1          1     0     0        0
```

ADVANCED | MINI PROJECT: SALES KPIS

Q149. Compute these KPIs from sales: total revenue, average order value, best product by revenue, best region, and the revenue growth between the first and last month.

CONCEPTS USED: assign, groupby, agg, idxmax, pct_change

SOLUTION

```
df = sales.assign(rev=sales['qty'] * sales['price'])
monthly = df.groupby(df['order_date'].dt.to_period('M'))['rev'].sum()
print('Total revenue :', df['rev'].sum())
print('Avg order     :', round(df['rev'].mean(), 1))
print('Best product  :', df.groupby('product')['rev'].sum().idxmax())
print('Best region   :', df.groupby('region')['rev'].sum().idxmax())
print('Growth first->last month: %.1f%%' % ((monthly.iloc[-1] / monthly.iloc[0] - 1) * 100))
```

OUTPUT

```
Total revenue : 434220
Avg order     : 36185.0
Best product  : Laptop
Best region   : West
Growth first->last month: -79.0%
```

ADVANCED | MINI PROJECT: HR REPORT

Q150. Create a department report with headcount, average salary, average experience, the highest-paid employee and the department's salary rank.

CONCEPTS USED: groupby, agg, rank, merge, round

SOLUTION

```
g = emp.groupby('dept')
rep = g.agg(headcount=('emp_id', 'count'), avg_salary=('salary', 'mean'), avg_exp=('exp', 'mean'))
rep['top_earner'] = emp.loc[g['salary'].idxmax()].set_index('dept')['name']
rep['salary_rank'] = rep['avg_salary'].rank(ascending=False).astype(int)
print(rep.round(1).sort_values('salary_rank'))
```

OUTPUT

```
         headcount  avg_salary  avg_exp top_earner  salary_rank
dept
Finance          2     75500.0     10.0      Meera            1
IT               3     68666.7      3.0      Kabir            2
HR               2     61500.0     13.5       Neel            3
Sales            3     52333.3      8.3     Vikram            4
```

Keep Practicing!

You have completed 150 Pandas problems. Next, move on to data visualization with Matplotlib and Seaborn.

SKILLARGER - Learn. Build. Get Hired.